Assignment: Multithreaded Magic Square Validator (MMSV)

Course: Operating Systems – COMP2006 Semester 1 / Trimester 1A – 2026

Weight: 15% of Final Grade

Language: C

Due date: Monday 27 April 2026, Time: 11.59PM AWST

1. Objective

The primary objective of this assignment is to provide hands-on experience with multithreading and inter-thread communication. You will learn how to:

- Manage shared resources in a concurrent environment.
- Protect critical sections using Mutexes.
- Coordinate thread execution and signalling.

2. Assignment Description

A Magic Square is an $n \times n$ grid of distinct integers from 1 to n^2 such that the sum of the numbers in each row, each column, and both main and secondary diagonals are all equal to the same value, known as the magic constant.

For further mathematical background on magic squares and the calculation of the magic constant, refer to: https://mathworld.wolfram.com/MagicSquare.html

Example: A valid 3 x 3 Magic Square

8	1	6
3	5	7
4	9	2

- **Row Sums:** 8+1+6=15, 3+5+7=15, 4+9+2=15
- **Column Sums:** 8+3+4=15, 1+5+9=15, 6+7+2=15
- **Diagonal Sums:** 8+5+2=15, 6+5+4=15
- **Uniqueness:** Contains all numbers from 1 to 9 exactly once.

3. Requirements

You are required to write a C program that validates whether a matrix provided in a text file is a valid magic square. Your program must use at least four concurrent threads in addition to the main (parent) thread. While you are free to design the thread function yourself, the following tasks must be performed concurrently:

- Row, Column and Diagonal validation: Verify that the sum of every individual row, column and diagonal equals magic constant.

- Uniqueness Validation**:** Ensure the grid contains every integer from 1 to n^2 exactly once.

Concurrency & Synchronization

To receive full marks, your implementation must adhere to these architectural constraints:

1. Thread Management: The main thread is responsible for reading the input file, spawning the workers, and printing the final report. But no validation calculations itself.
2. Variables to maintain:
   - Implement a persistent data structure (e.g., arrays or status flags) to record the pass/fail result of every individual row, column, diagonal, and the uniqueness check. Each worker thread must update the entries corresponding to its tasks, allowing the main thread to traverse these records and print a comprehensive validation report after all threads have completed.
   - Global score variable**:** Maintain a global shared counter that tracks the total points of the square.
     1. Every time a row, column, diagonal, or the uniqueness check is confirmed valid, the responsible thread must increment this score.
     2. If the validation is failed, decrease the `score` by 1. Minimum value for `score` is 0.
     3. Only one thread should update the `score` at a time.
     4. See the example below for n =3. `score` = 0 initially

Component	Valid/invalid	Change	`score` value
Initial state	-	-	0
Row 1	Valid	+1	1
Row 2	Valid	+1	2
Row 3	Invalid	-1	1
Column 1	Valid	+1	2
Column 2	Valid	+1	3
Column 3	Valid	+1	4
Diagonal	Invalid	-1	3
Uniqueness	Invalid	-1	2

3. You need to implement a proper synchronization between the threads, and the main thread should wait for all processes to finish.
4. Artificial Delay: Each worker thread should include a small sleep before updating shared results to simulate heavy processing and test your synchronization robustness.

Input & Output

1. Input: The program must accept a filename via command-line arguments. The file will contain the size n followed by the $n \times n$ matrix.
   Example: for n =3
   3
   8 1 6
   3 5 7
   4 9 1

2. **Output:** Upon completion, the main thread must print a detailed validation report showing the following components.
    1. **Thread Completion Logs:**
        - `Thread ID-[TID] completed: Rows`
        - `Thread ID-[TID] completed: Columns`
        - `...`
    2. **Status Summary:**
        - List specifically shows which rows/columns/diagonal or uniqueness failed if the square is invalid.
        - `Final Score: (score) / (maximum allowable score)`
    3. **Final Verdict:**
        - `RESULT: VALID MAGIC SQUARE or RESULT: INVALID MAGIC SQUARE`

A sample console output:

Valid Case Example (n=3)	**Invalid Case Example (n=3)** **(A duplicate number is at 3,3 location)**

`Thread ID-2: Column checks completed.`
`Thread ID-4: Uniqueness check completed.`
`Thread ID-3: Diagonal checks completed.`
`Thread ID-1: Row checks completed.`
 	
`--- Magic Square Report ---`
`Rows:  All Valid`
`Cols:  All Valid`
`Diags:  All Valid`
`Unique: Passed`
`Final Score:  9 / 9`
`RESULT: VALID MAGIC SQUARE`
	`Thread ID-1: Row checks completed.`
	`Thread ID-3: Diagonal checks completed.`
	`Thread ID-4: Uniqueness check completed.`
	`Thread ID-2: Column checks completed.`
	
	`--- Magic Square Report ---`
	`Rows:  Row 3 Invalid`
	`Cols:  Col 3 Invalid`
	`Diags:  Main Diag Invalid`
	`Unique: Failed (Duplicates found)`
	`Final Score:  1 / 9`
	`RESULT: INVALID MAGIC SQUARE`

4. Instruction for submission

1. Assignment submission is **compulsory**. As stated in the unit outline, the penalty for late submission is calculated as follows:

• *For assessment items submitted within the first 24 hours after the due date/time, students will be penalised by a deduction of 5% of the total marks allocated for the assessment task,*

• *For each additional 24 hour period commenced an additional penalty of 10% of the total marks allocated for the assessment item will be deducted, and*

• *Assessment items submitted more than 168 hours late (7 calendar days) will receive a mark of zero.*

2. Please read and follow the Academic Integrity (including plagiarism and cheating) section in the unit outline to prevent any possible academic misconduct.

3. You must submit .zip file to the BB including the (i) soft copy of the report and (ii) your program files, i.e., mmsv.c and other files including makefile, test input and output files, readme file that explains how to compile your program and how to run the program.

4. Your code files should adhere to

i) Proper in-line and header documentation

ii) Meaningful variable names and delete all unnecessary comments

5. You will demonstrate your code and its execution to your tutor/instructor online (MS Teams) and this is **mandatory**. You should demonstrate and run the same code you uploaded to the BB. Further instructions regarding time slots and demonstration will be informed later.

6. Your assignment **report** should include:

1. A signed Declaration of Originality form (available in the unit Blackboard). Please read the form carefully before you sign it. Your name should be that as recorded in the student database. By signing the form, among others, you agree on the following two statements:
   a. *The work I am submitting is entirely my own, except where clearly indicated otherwise and correctly referenced.*
   b. *Even with correct referencing, my submission will only be marked according to what I have done myself, specifically for this assessment. I cannot re-use the work of others, or my own previously submitted work, in order to fulfil the assessment requirements.*

2. Introductory summary of your implementation (e.g., Problem you have solved, your solution in brief). This will be half-page description.

3. Detailed discussion on how synchronization is achieved when accessing shared resources / variables and which threads access the shared resources.

4. Description of any cases for which your program is not working correctly or how you test your program that makes you believe it works perfectly.

5. Sample inputs (for different n values) and outputs from your running programs. Explain if the outputs are correct / incorrect.

Your report will be assessed (worth 20% of the overall <u>assignment</u> mark).